

01. 시설팀 — 원내 공간 조화·관리 C 조화·관리

❖ 이 페이지 전제 — 복불 전 1초 확인 (Sonnet도 그대로 동작하게 다 적었습니다) - 답변(채팅) 모델: **qwen3.6:35b-a3b** · 호출 **POST**
http://121.138.151.6:11434/api/chat - (참고) 검색(임베딩) 모델 **bge-m3:latest** 은 다음 강의용 — 이번 숙제에서는 쓰지 않습니다(자료를 통째로 모델에 넘김). - 내 백엔드: FastAPI
http://127.0.0.1:8000 — 프론트(HTML)는 반드시 이 백엔드만 부릅니다. - ⚡ 금지: HTML/자바스크립트에서 **11434** · **ollama** 직접 호출 금지(CORS·보안 사고). 모델 호출은 오직 백엔드(8000) 경유. - 출발점: 공통 STEP 0 기본코드(**app.py** · **index.html**)가 떠 있는 상태에서 시작(이 페이지에 코드를 다시 만들지 않음).

한 줄: 원내 공간을 자연어로 묻는 챗봇. “○○과 어디 있어?”, “면적 가장 큰 부서는?” 처럼 말로 묻고, 백엔드가 내 공간 자료(샘플 8칸)를 근거로 출처와 함께 답합니다. 이번 주 목표: 질문 입력창 + 답변창(+출처)이 동작하는 자료 기반 챗봇 1개. 표·CSV는 보조로 남깁니다.

ℹ 이 숙제는 임베딩이 없습니다(다음 강의에서 임베딩 검색을 붙입니다). 대신 내 공간 자료(8칸)를 통째로 컨텍스트에 넣어(stuffing) 모델에게 “이 자료만 보고 답하라”고 시킵니다. 검색 알고리즘 없이도 자료 근거 답변·출처가 됩니다. 숫자 집계 질문(면적 합 최대 부서 등)은 백엔드가 파이썬으로 정확히 계산한 값을 컨텍스트에 같이 넣어, 모델이 그 값을 인용해 답하게 합니다(모델이 숫자를 지어내지 않게 — 환각 방지).

💡 읽는 법: 아래 프롬프트를 번호 순서대로 하나씩 Claude Code에 복붙하세요. 한 번에 다 넣지 말고, 하나 끝나고 화면 확인 → 다음 프롬프트. 공통 프롬프트(①②③)를 먼저 끝낸 뒤 시작합니다.

📦 공통 프롬프트(①②③)가 미리 만들어 주는 것 (이게 떠 있어야 아래가 복불만으로 동작합니다): - ① 백엔드 FastAPI **app.py** (**http://127.0.0.1:8000**) — **GET /health** 엔드포인트와 CORS 허용(프론트가 :8000을 부를 수 있게)이 포함됩니다. - ② 프론트 **index.html** 기본 골격 — 화면이 :8000 백엔드를 호출하는 빠대. - ③ 실행 **uvicorn** 으로 :8000 띄우기 + 브라우저로 **index.html** 열기. - 즉, 아래 프롬프트가 쓰는 **:8000** · **/health** · CORS는 공통 ①②③에서 이미 만들어진 것을 재사용합니다(이 페이지에서 새로 만들지 않음).

먼저 쓸 샘플 데이터 (복붙해 둘 것)

아래 8개 공간 행을 프롬프트 1에서 그대로 씁니다. 컬럼은 **공간코드** / **공간명** / **위치** / **면적** / **부서** / **구분**. 전부 더미·비식별입니다.

```
[{"공간코드": "B6-001", "공간명": "기계실", "위치": "본관 지하6층", "면적": 2296, "부서": "시설팀", "구분": "기계시설"}, {"공간코드": "B2-014", "공간명": "주차관제실", "위치": "본관 지하2층", "면적": 45, "부서": "시설팀", "구분": "주차시설"}, {"공간코드": "1F-002", "공간명": "외래접수", "위치": "본관 1층", "면적": 540, "부서": "원무팀", "구분": "진료지원"}, {"공간코드": "1F-010", "공간명": "직원식당", "위치": "본관 1층", "면적": 420, "부서": "영양팀", "구분": "급식시설"}, {"공간코드": "2F-021", "공간명": "약제부", "위치": "본관 2층", "면적": 259, "부서": "약제부", "구분": "약제시설"}, {"공간코드": "3F-105", "공간명": "제1세미나실", "위치": "별관 3층", "면적": 86, "부서": "교육팀", "구분": "교육시설"}, {"공간코드": "4F-210", "공간명": "인공신장실", "위치": "본관 4층", "면적": 312, "부서": "간호부", "구분": "진료시설"}, {"공간코드": "5F-301", "공간명": "의무기록보관실", "위치": "별관 5층", "면적": 168, "부서": "의무기록팀", "구분": "행정시설"}]
```

프롬프트 1 — 자료를 통째로 넣어 답하는 챗봇 (핵심 · stuffing)

 이 숙제의 핵심: 검색 알고리즘 없이, 공간 자료 8칸을 모두 컨텍스트 (시스템 프롬프트)에 넣고 “이 자료만 보고 한국어로 답하라”고 시킵니다. 자료에 없으면 “확인되지 않습니다”. 이게 ‘자료 기반 자연어 대화 챗봇’의 가장 단순한 형태입니다.

공통으로 만든 백엔드 app.py 에 두 가지를 추가해줘. 코드를 처음부터 다시 만들지 말고 거기에 얹어줘.

[1] GET /api/data

- 위에서 준 샘플 공간 데이터(8개)를 리스트 ROWS 로 둔다(딕셔너리 한 줄 = 공간 하나).
- {"mode":"demo", "rows": ROWS} 형태의 JSON으로 반환한다.

[2] POST /api/ask (← 이번 숙제의 핵심: 자료를 통째로 넣어 답하는 챗봇)

- {question:"..."} 을 받는다.
- ROWS(공간 8칸)를 사람이 읽기 쉬운 한 줄씩 텍스트로 만들어 컨텍스트(근거)로 쓴다.

예: "기계실 | 본관 지하6층 | 면적 2296제곱미터 | 시설팀 | 기계시설" 을 8줄.

- 그 8줄 전체를 system 프롬프트에 넣고, 채팅 모델로 답을 만든다:

```
POST http://121.138.151.6:11434/api/chat , body
{"model":"qwen3.6:35b-a3b",
 "messages":[
   {"role":"system","content":
     "너는 병원 공간 안내 챗봇이다. 아래 [공간자료] 8줄에
     적힌 사실만 근거로 한국어로 간결히 답한다. "
     "면적·위치·부서·구분은 자료의 값 그대로 쓴다. 자료에
     없는 건 절대 지어내지 말고 "
     "'자료에서 확인되지 않습니다'라고만 답한다.\n\n[공간자
     료]\n" + 위에서 만든 8줄 텍스트},
   {"role":"user","content": question}],
  "stream": false}
```

→ 응답 message.content 를 답으로 쓴다.

- 응답은 {answer:"...", sources:[해당 행들]} JSON으로 한 번에 준다(SSE 아님). sources 는 답에 쓴 공간 행 그대로.

[중요] 프론트(index.html)는 절대 121.138.151.6:11434 나 ollama 를 직접 호출하지 말 것.

챗봇 요청은 반드시 내 백엔드 http://127.0.0.1:8000 의 POST /api/ask 로만 보낸다. 모델 호출은 오직 app.py 안에서.

index.html 도 수정해줘.

- 화면 맨 위에 '공간 챗봇' 영역: 질문 입력창 1개 + [보내기] 버튼 + 답변 영역.
- placeholder: "예: 약제부 어디 있어? / 본관 1층에 뭐 있어? / 시설팀 공간 알려줘".
- [보내기]를 누르면 /api/ask 로 question 을 보내고, 받은 answer 를 답변 영역에 보여준다.
- 데이터를 브라우저 코드에 직접 박지 말 것. 모델 호출도 프론트에서 하지 말 것(반드시 :8000 경유).

기대 결과: “약제부 어디 있어?”라고 입력하면 챗봇이 “**약제부는 본관 2층에 있습니다(면적 259제곱미터, 약제부 소관)**” 류로 자료 근거로 답한다. “수영장 어디?” 같은 무관한 질문엔 “**자료에서 확인되지 않습니다**”. 복불

→ **화면 확인**: 입력창에 질문을 넣고 보내기 → 답변 영역에 **자료 기반 한국어 답**이 뜨면 성공. 개발자도구 Network 탭에 **:8000/api/ask** 호출만 보이고 **11434** 는 안 보이면 분리 정상.

프롬프트 2 — 출처 표시 (어떤 자료를 보고 답했나)

챗봇 답변 아래에 '출처(근거)'를 함께 보여줘.

- 백엔드 /api/ask: 답에 사용한 공간 행들을 sources 배열로 그대로 담는다.
예: `sources: [{"공간명": "약제부", "위치": "본관 2층", "면적": 259, "부서": "약제부"}]`.
(질문에 등장한 부서·공간명·위치 키워드가 들어간 행을 sources 후보로 고르고, 없으면 빈 배열.)
- index.html: 답변 아래에 작게 "근거: 약제부(본관 2층, 259 제곱미터)" 처럼 출처 칩을 표시한다.
- 출처는 ROWS 원본 행 그대로다(모델이 새로 지어낸 이름이 아니라). 답과 출처가 같은 행을 가리키게.
-  프론트는 계속 `http://127.0.0.1:8000` 만 호출.
`11434·ollama` 직접 호출 금지.

기대 결과: 답 밑에 “근거: ○○○(위치, 면적)” 칩이 떠서, 챗봇이 어떤 공간 자료를 보고 답했는지 눈으로 확인된다. **복붙** → **화면 확인**: “약제부 어디?” 답 아래에 “근거: 약제부(본관 2층…)” 칩이 보이면 성공.

프롬프트 3 — 집계·정확대조 질문은 백엔드 계산값을 근거로 (환각 방지·핵심)

 **여기가 숫자 환각을 막는 부분**. “면적 가장 큰 부서?” 같은 집계 질문은 모델에게 숫자를 시키지 않습니다. **백엔드가 파이썬으로 정확히 계산한 값을** 컨텍스트에 넣고, 모델은 그 값을 **인용해서 문장으로만** 답하게 합니다.

/api/ask 를 이렇게 보강해줘. (엔드포인트는 그대로 POST /api/ask 유지.)

질문을 받으면, 답변 모델을 부르기 전에 백엔드(파이썬)가 먼저 '집계 근거값'을 계산해서 컨텍스트에 같이 넣는다.

백엔드가 매 질문마다 ROWS 로 계산해 둘 값(결정론적):

- 면적이 가장 큰 공간 1개: ROWS 를 면적 내림차순 정렬한 1위 (공간명·면적).
- 면적이 가장 작은 공간 1개: 정렬 마지막 (공간명·면적).
- 부서별 면적 합계: 부서별로 면적을 더해 큰 순으로 정렬한 목록.
- 전체 공간 수, 전체 면적 합계, 평균 면적.

이 계산 결과를 사람이 읽는 한 줄들로 만들어 system 프롬프트에 [집계근거] 블록으로 추가한다. 예:

" [집계근거]

면적 최대 공간: 기계실 2296제곱미터

면적 최소 공간: 주차관제실 45제곱미터

부서별 면적 합계(큰 순): 시설팀 2341, 외래(원무팀) 540, 영양팀 420, 간호부 312, 약제부 259, 의무기록팀 168, 교육팀 86

전체 8칸, 합계 4126제곱미터, 평균 515.8제곱미터"

system 프롬프트에 규칙 추가: "숫자(면적·합계·순위)는 위 [집계 근거]에 적힌 값만 그대로 인용한다. 직접 계산하거나 지어내지 마라."

→ 모델은 이 값을 문장으로 풀어 답하기만 한다.

-  모델·임베딩 호출은 모두 백엔드(:8000)에서만. 프론트는 :8000/api/ask 만 부른다.

기대 결과(검증 가능한 결정론적 정답값): - “면적이 가장 큰 공간은?” → **기계실(2296제곱미터)** - “면적 합이 가장 큰 부서는?” → **시설팀(2341제곱미터 = 기계실 2296 + 주차관제실 45)** - “전체 면적 합계는?” → **4126제곱미터** · “평균 면적은?” → **약 515.8제곱미터** 이 값들은 백엔드 집계라 누가 언제 물어도 똑같이 나온다(모델이 숫자를 지어내지 않음). **복불 → 화면 확인:** “면적 합이 가장 큰 부서는?”에 챗봇이 **시설팀, 2341**을 답하면 성공. (백엔드 집계값을 그대로 인용한 것.)

프롬프트 4 — 보조 표 + 집계카드 + CSV (조회는 챗봇의 보조로 남김)

챗봇이 중심이고, 표·CSV는 **눈으로 거르는 보조**입니다. 같은 ROWS·같은 백엔드 데이터를 봅니다.

index.html 의 챗봇 아래쪽에 '보조: 공간 목록' 영역을 추가해줘 (챗봇이 위, 표가 아래).

- GET /api/data 를 불러와 공간 8개를 표(table)로 보여준다. 컬럼: 공간코드 / 공간명 / 위치 / 면적(제곱미터) / 부서 / 구분.
- 표 위에 검색창 1개(공간명·위치 부분검색)와 드롭다운 2개(부서·구분, 값 자동 채움). '초기화' 버튼으로 리셋. 필터는 즉시 반영.
- 표 위에 집계카드 3개(지금 표에 보이는 행 기준): 공간 개수 / 면적 합계(천단위 콤마) / 평균 면적(소수 첫째).
- 'CSV 다운로드' 버튼: 지금 보이는 행만 CSV 저장(헤더 포함, 한글 안 깨지게 UTF-8 BOM).
- 데이터는 계속 /api/data 에서 받은 것만 쓴다. 프론트는 :8000 만 호출.

기대 결과: 위쪽 챗봇은 말로 묻고, 아래쪽 보조 표는 “약” 검색·부서=시설 팀 필터로 좁혀지며 카드 숫자가 따라 바뀐다. CSV 버튼으로 보이는 행이 엑셀로 열 수 있게 저장된다. **복붙 → 화면 확인:** 챗봇(위)과 표·카드·CSV(아래)가 한 화면에 같이 보이고, 표 필터를 바꾸면 카드 숫자가 따라 변하면 성공.

프롬프트 5 — 에러·빈값·안전 처리 (안정화)

index.html 과 app.py 를 더 튼튼하게 만들어줘.

- 1) 백엔드가 안 켜져 있으면 화면에 빨간 글씨로 "백엔드(:8000)가 꺼져 있어요. 터미널에서 실행하세요" 안내.
- 2) 챗봇 질문이 비어 있으면 전송하지 않는다.
- 3) 자료에 없는 걸 물으면(예: "비상금고 위치") 챗봇이 지어내지 말고 "자료에서 확인되지 않습니다"라고만.
- 4) 보안·시설 민감정보(잠금번호, 상세 도면, 출입통제 절차)를 물으면 "보안시설 상세는 안내하지 않습니다"로 정중히 거절.
- 5) 표 필터 결과가 0건이면 "조건에 맞는 공간이 없습니다" 문구. 면적이 비었거나 숫자가 아니면 0으로 처리(합계 안 틀리게).
- 6) 화면 위에 'Demo Mode (샘플 공간 데이터)' 배지를 항상 표시.

🚫 어느 처리든 프론트는 http://127.0.0.1:8000 만 부른다. 11434·ollama 직접 호출 금지.

기대 결과: 백엔드를 끄면 안내 문구, 자료 밖 질문엔 “확인되지 않습니다”, 보안 질문엔 거절 문구, 표 0건엔 안내 문구가 뜬다. **복붙 → 화면 확인:** “비상금고 위치”에 “확인되지 않습니다”, 보안 질문에 거절 문구가 뜨면 성공.

프롬프트 6 — 막혔을 때: 디버깅 요청 (그대로 복붙)

챗봇 답이 안 나오거나, 출처가 안 뜨거나, 집계 숫자가 이상할 때 아래를 복붙하세요.

지금 내 공간 챗봇이 제대로 안 돼. 아래를 순서대로 점검하고 고쳐 줘.

- 1) 백엔드가 떠 있는지: 브라우저에서 `http://127.0.0.1:8000/health` 와 `/api/data` 가 정상인지 확인.
- 2) 챗봇이 답을 못 하면: 백엔드가 호출하는 채팅 모델 서버 (`qwen3.6:35b-a3b @ POST http://121.138.151.6:11434/api/chat`) 연결을 백엔드에서 확인. (프론트가 11434를 직접 부르고 있지 않은지도 함께 점검 - 호출은 모두 :8000 경유여야 함.)
- 3) 출처(sources)가 화면에 안 뜨면, `/api/ask` 응답 JSON에 sources 배열이 들어오는지와 프론트의 출처 렌더 부분을 대조.
- 4) 집계 답(면적 최대·부서 합계)이 이상하면, 백엔드 [집계근거] 계산값과 표(`/api/data`)를 대조 - 숫자는 반드시 백엔드 계산값을 인용해야 한다(모델이 지어내면 안 됨).
- 5) CORS 오류가 보이면 백엔드 CORS 설정을 점검. 무엇이 문제였는지 한국어로 쉽게 알려주고, 고친 부분을 요약해줘.

기대 결과: 막힌 원인을 사람 말로 알려주고, 고친 곳을 짚어 챗봇이 다시 답한다.

프롬프트 7 — 기능 1개 추가 + 자동 검증(Playwright)

두 가지를 추가해줘.

- 1) 보조 표 헤더(면적, 공간명)를 클릭하면 그 기준으로 오름차순·내림차순 정렬(다시 누르면 방향 바뀜).
추가로 '부서별 면적 합계'를 백엔드 [집계근거]의 값으로 큰 순서대로 가로 막대(div 너비)로 표 아래에 보여줘.
- 2) Playwright로 자동 테스트를 만들어줘 (`tests/qa.spec.ts`).
 - 화면(`index.html`)을 열고 백엔드(:8000) 연결 확인 (`http://127.0.0.1:8000/health` 정상).
 - 챗봇 입력창에 "약제부 어디 있어?" 입력 후 보내기 → 5초 안에 답이 뜨고, 답에 '약제부'·'본관 2층'이 포함되고 '근거'(출처)가 표시되는지 확인.
 - 챗봇에 "면적이 가장 큰 공간은?" 입력 → 답에 '기계실'과 '2296'이 포함되는지 확인(백엔드 집계 - 결정론적 정답).
 - 챗봇에 "면적 합이 가장 큰 부서는?" 입력 → 답에 '시설팀'과 '2341'이 포함되는지 확인(백엔드 집계 - 결정론적 정답).
 - 프론트가 121.138.151.6:11434 를 직접 부르지 않고 오직 `http://127.0.0.1:8000(/api/ask, /api/data)` 만 호출하는지 네트워크 로그로 점검.
 - 통과/실패를 리포트하고, 실행 방법(`npm playwright install` 포함)도 알려줘.

기대 결과(검증 가능한 정답값): 챗봇 “면적이 가장 큰 공간은?” → **기계실(2296제곱미터)**, “면적 합이 가장 큰 부서는?” → **시설팀(2341제곱미터 = 기계실 2296 + 주차관제실 45)**. 이 두 값은 백엔드 집계로 **항상 똑같이** 재현된다(모델·임베딩과 무관). “Playwright로 테스트해줘”만으로 자동 점검이 통과로 뜬다. **복붙 → 화면 확인:** 터미널에 Playwright 리포트가 뜨고 모든 케이스가 **passed**로 표시되면 성공.

✅ 여기까지 쓰면 숙제 성공. 핵심은 자료를 통째로 넣어 답하는 챗봇(질문창+답변창+출처) 입니다. 집계 질문은 백엔드 계산값을 근거로 인용해 환각을 막았습니다. 표·CSV는 보조입니다. ⓘ 다음 강의 연결: 강의에서는 이 챗봇의 '통째로 넣기(stuffing)'를 임베딩 검색(bge-m3) 으로 바꿔 관련 조각만 골라 근거로 답하고, 바이브↔하네스 토글을 엮었습니다. 강의 STEP 0은 공통 기본코드(`POST /api/chat` SSE 스트림 @ 모델 `qwen3.6:35b-a3b`)가 떠 있는 상태에서 출발합니다. 안전: 더미 데이터만 사용 · `Demo Mode` 표시 · 보안시설 상세(도면·잠금·출입통제) 는 입력·표시·안내 금지 · 모델 서버 주소(`121.138.151.6:11434`) 는 외부 공유 금지 · 🚫 HTML/JS에서 `11434` · `ollama` 직접 호출 금지(채팅 호출은 모두 :8000 백엔드 경유).